

CSC 4553

Fall 2026 - Day 7

Admin

- Programming assignment 1 due Wednesday, 18:00

Process communication

Questions to consider

- What are the two main strategies for IPC?
- How do they differ?
- In what situations would you choose one over the other?

Processes give us a protection boundary

- The operating system is responsible for isolating processes from each other
- What you do in your own process is your own business *but it shouldn't be able to crash the machine or affect other processes*, or at least processes started by other users
- Thus: safe intra-process communication is **your** problem; safe inter-process communication is an **operating system** problem

Why do we need IPC? What are the benefits?

- **Data Sharing:** IPC allows processes to share data efficiently, which is crucial for applications requiring real-time data exchange
- **Modularity:** It promotes modularity by enabling different parts of a system to communicate, making the system easier to manage and scale
- **Resource Utilization:** IPC can help optimize resource utilization by allowing processes to coordinate their use of shared resources
- **Concurrency** (scalability): It supports concurrent execution of processes, improving the overall performance and responsiveness of applications

What are the disadvantages of IPC?

- **Complexity:** Implementing IPC can add complexity to the system, requiring careful design and management to avoid issues like deadlocks and race conditions
- **Overhead:** IPC mechanisms can introduce overhead, potentially impacting performance, especially if the communication is frequent or involves large amounts of data
- **Security:** Ensuring secure IPC can be challenging, as it involves protecting data from unauthorized access and ensuring the integrity of the communication
- **Debugging:** Debugging IPC-related issues can be difficult, as problems may arise from interactions between multiple processes, making them harder to isolate and resolve

What are the two main categories of IPC?

- **Message passing**

- High-level abstraction for exchanging packets of information over some interconnect

- **Shared memory**

- Region of memory available to different processes; writable by at least one process

Message passing

- Kernel establishes and oversees all communication
 - Process copies data to buffer, then issue system call to request transfer
 - Kernel copies data into its memory
 - Later, process issues system call to retrieve
- Two primitives: `send()` and `recv()`
- Beyond intra-computer communication, facilitates processes over a network; link implementation is unimportant

Pros and cons of message passing?

- Pros:
 - **Easier** to implement and manage, especially in distributed systems
 - Provides clear boundaries between processes, enhancing security and **modularity**
- Cons:
 - Can introduce **overhead** due to the need for message formatting and transmission
 - May be slower compared to shared memory for large volumes of data

Shared memory

- Kernel plays a role in establishing and attaching the address space, but does not control read/write access beyond that
- How the memory is shared, and kept consistent, is left up to the processes

Pros and cons of shared memory?

- Pros:
 - Offers **high-speed** data exchange, as processes can directly read and write to the shared memory
 - **Efficient** for large volumes of data
- Cons:
 - Requires careful **synchronization** to avoid conflicts and ensure data consistency
 - Can be more **complex** to implement and debug

What isn't clear?

Comments? Thoughts?

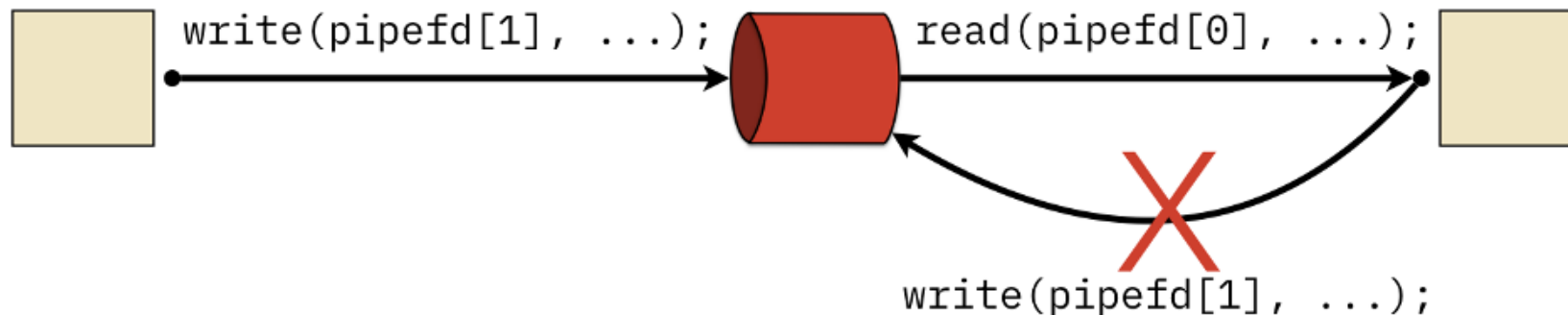
IPC mechanisms

Questions to consider

- How do pipes differ from FIFOs and when would you use each?
- What considerations matter when choosing between pipes, FIFOs, and memory-mapped files?

Pipes (anonymous pipes)

- Pipes are **unidirectional**; one end must be designated as the reading end and the other as the writing end
- Pipes are **order preserving**; all data read from the receiving end of the pipe will match the order in which it was written into the pipe
- Pipes have a **limited capacity** and they use **blocking I/O**; if a pipe is full, any additional writes to the pipe will block the process until some of the data has been read
- Pipes send data as unstructured **byte streams**. There are no pre-defined characteristics to the data exchanged, such as a predictable message length



Pipes (cont'd)

- Pipes create a producer-consumer buffer between two processes
- Cannot be accessed outside of the creating process (child inherits the pipe since it's a fd)
- The operating system manages a queue for each pipe to accommodate different input and output rates
- Facilitates the canonical chaining together of small UNIX utilities to do more sophisticated processing

```
prs@unix1:~$ ps aux | grep prs
root      11712  0.0  0.0 193444  5828 ?        Ss   10:24   0:00 sshd: prs [priv]
prs       11735  0.0  0.0 193444  2400 ?          S    10:24   0:00 sshd: prs@pts/0
prs       11736  0.0  0.0 126252  2152 pts/0    Ss+  10:24   0:00 -bash
root      15645  0.0  0.0 193444  5832 ?        Ss   11:11   0:00 sshd: prs [priv]
prs       15660  0.0  0.0 193444  2544 ?          S    11:11   0:00 sshd: prs@pts/4
prs       15661  0.0  0.0 126376  2240 pts/4    Ss   11:11   0:00 -bash
prs       16202  0.0  0.0 166156  1928 pts/4    R+   11:16   0:00 ps aux
prs       16203  0.0  0.0 112812   976 pts/4    S+   11:16   0:00 grep --color=auto prs
```

Pipe bugs

```
1 int pipefd[2];
2 pipe (pipefd);
3
4 if (fork () == 0)
5     exit (0);
6
7 char buffer[10];
8 read (pipefd[0], buffer, sizeof (buffer));
```

- What will happen here?
 - Parent will block indefinitely. Since the child exits without writing anything, and the parent has not closed its own write end, the read end never sees **EOF** (**EOF** is only signaled when all write ends are closed).

Named pipes (also known as FIFOs)

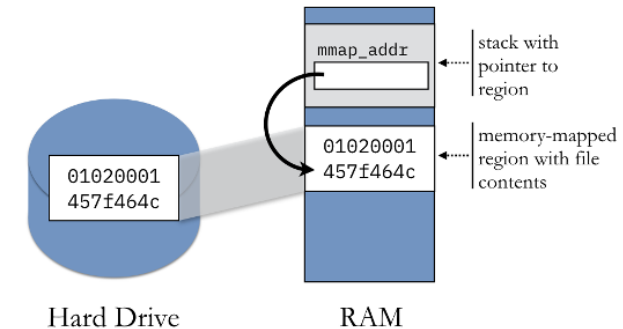
- Of course there is another type that don't share the same characteristics
- Named pipes don't require a parent-child relationship
 - Creates a persistent file-like name
 - Require a reader and writer
 - Can be used for unrelated processes to communicate

Named pipes (cont'd)

- Really a data stream (ordering, buffering, reliability, authentication all implied)
- They are **not** regular files, though they look like them
 - Once data has been read from a FIFO, the data is discarded and cannot be read again
 - Cannot broadcast: only one `read()`
 - **Not** wise to use bidirectionally, why?

Shared memory with mmap

- Memory-mapped files allow for multiple processes to share read-only access to a common file. Example: `libc.so` mapped into all running C programs
- Data is accessed directly from the page cache without an extra copy into a user buffer
- Provide extremely fast IPC data exchange. i.e., when one process writes to the region, that data is immediately accessible by the other process **without having to invoke a system call**
- Unlike pipes, memory-mapped files create **persistent** IPC. Once the data is written to the shared region, it can be repeatedly accessed by other processes



Memory mapped files: POSIX and System V

- These two ultimately **mmap** a file, but do some trickery beforehand and use special files
- POSIX == named, RAM-backed file descriptors
 - **name** → **fd** → **mmap**
 - usually backed by **tmpfs** (**/dev/shm**)
- System V shared memory
 - Predates POSIX - designed before file descriptors were a universal abstraction
 - Kernel-persistent objects, you must clean up manually
 - Process exits **≠** memory freed

POSIX vs. System V

- POSIX is great, right!?
 - POSIX (e.g., `shm_open()`) IPC may not work in your favor in macOS
 - Some questionable engineer (obviously not a CP grad) decided to provide the header files for certain things, but they are empty stubs
 - It will compile, but could be bad
- System V is “fine”
 - `shmget()`: allocate
 - `shmat()`: attach
 - `shmdt()`: detach
 - `shmctl()`: control (destroy with `IPC_RMID`)

Message passing vs. shared memory

A camera captures frames at 30fps and a separate process runs AI inference on each frame. **Which IPC mechanism would you use? What happens if inference is slower than capture?**



“Gemini, make an image in the style of a video game pitting pipes versus shared memory”

Demo

What isn't clear?

Comments? Thoughts?